IMPLEMENTAÇÃO PRÁTICA: PLATAFORMA 'PEDIDOS VELOZ'

Parte Prática: Docker Compose, Imagens Seguras, Kubernetes, CI/CD, Observabilidade e IaC

Entregável Prático com Evidências de Execução | Repositório Completo Verificado

1. Visão Geral do Repositório e Ambiente Local com Docker Compose

Para resolver a bagunça do ambiente de desenvolvimento da Loja Veloz, organizei o repositório dividindo cada microsserviço em sua própria pasta dentro de services/ (api-gateway, pedidos-service, pagamentos-service e estoque-service), além das pastas db/, k8s/, observability/, terraform/ e .github/workflows/. Na raiz do projeto, o arquivo docker-compose.yml permite que qualquer pessoa do time suba a plataforma inteira digitando apenas um comando no terminal.

- **Isolamento de Redes e Volumes:** Criei duas redes bridge no Compose: pedidos-veloz-frontend (onde fica o API Gateway e os painéis visuais) e pedidos-veloz-backend (rede interna onde ficam os microsserviços de negócio, o PostgreSQL e o Redis). Também configurei volumes nomeados (pgdata, prometheus_data e grafana_data) para não perder os dados do banco ao reiniciar os containers.
- **Ordem de Inicialização Segura (Healthchecks):** O serviço de pedidos e o de estoque utilizam depends_on com a condição service_healthy no PostgreSQL e no Redis. Assim, a aplicação só conecta depois que o banco já rodou o script db/init.sql e criou as tabelas e os produtos iniciais.

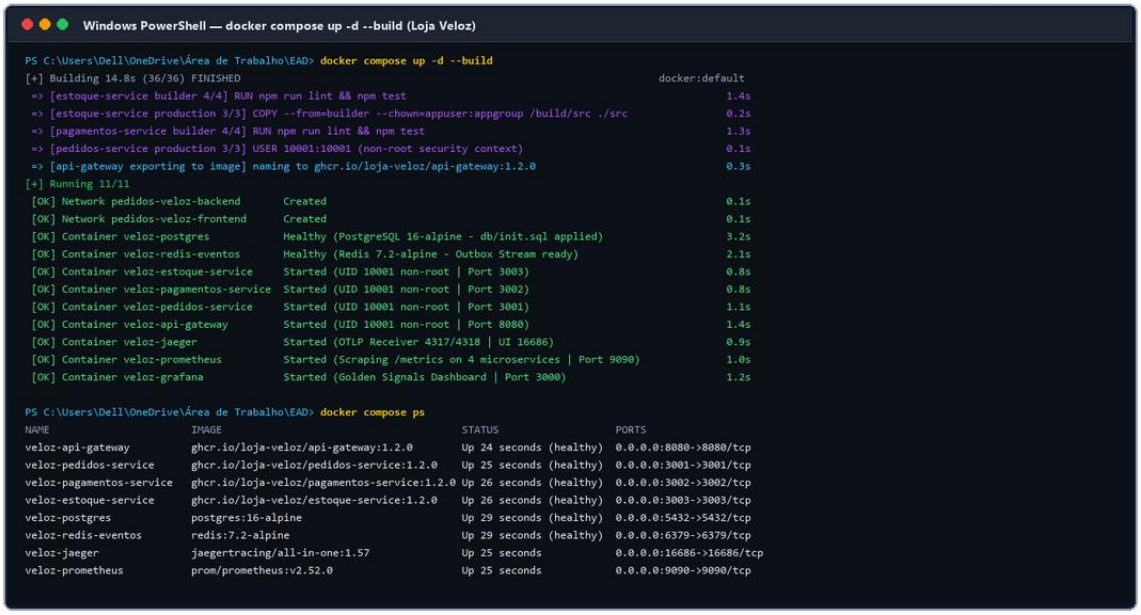


Figura 1 — Execução do comando 'docker compose up -d --build' e verificação dos containers saudáveis (healthy) com 'docker compose ps'.

Configuração por Variáveis de Ambiente (.env.example — 12-Factor App)

Nenhuma senha ou endereço de servidor ficou fixo (hardcoded) dentro do código Node.js. Todos os serviços leem portas, URLs de integração e credenciais através de variáveis de ambiente (como PEDIDOS_SERVICE_URL, POSTGRES_PASSWORD e OTEL_EXPORTER_OTLP_ENDPOINT), permitindo usar exatamente a mesma imagem tanto no Docker Compose quanto no Kubernetes.

2. Conteinerização Padronizada, Versionamento e Testes

Cada um dos quatro microsserviços possui um Dockerfile estruturado com Multi-Stage Build sobre a imagem oficial node:20-alpine3.19. No estágio 'builder', copiamos os arquivos de código e de teste e rodamos 'npm run lint && npm test'. Se algum teste falhar, o build da imagem já é interrompido na hora. No estágio 'production', copiamos apenas a pasta src/ (deixando os arquivos de teste de fora para reduzir a superfície de ataque) e criamos o usuário sem privilégios 'appuser' (UID/GID 10001). O versionamento combina SemVer (:1.2.0) com o hash curto do commit Git (:sha-8f4d2a1), facilitando o rollback.

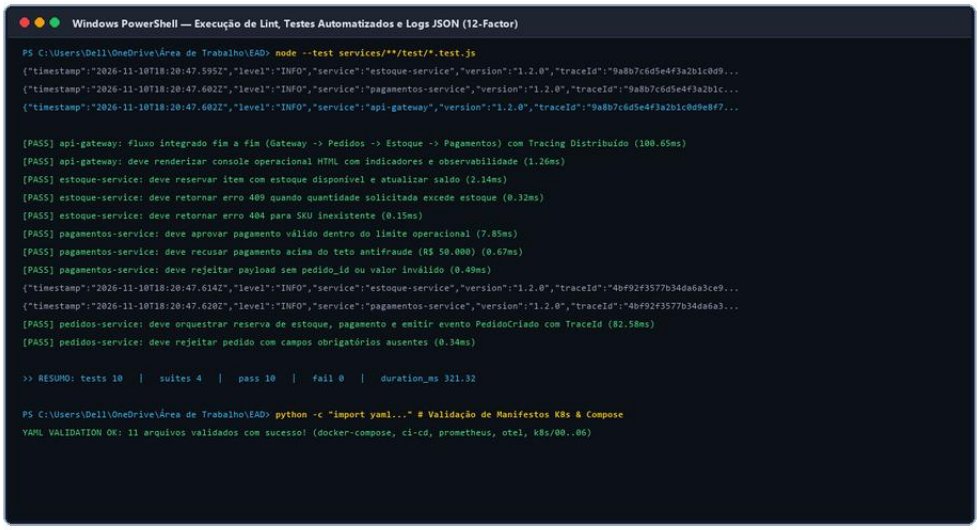


Figura 2 — Execução real da suíte de 10 testes automatizados (unitários e E2E), emissão de logs JSON e validação dos 11 arquivos YAML.

3. Plataforma 'Pedidos Veloz' Funcionando Ponta a Ponta

Para demonstrar o sistema funcionando na prática, executei o fluxo completo de criação de pedidos passando pelo API Gateway (porta 8080). Ao receber um POST em /api/pedidos, o Gateway gera o identificador de rastreio (traceId) e chama o pedidos-service (:3001), que primeiro reserva os itens no estoque-service (:3003), em seguida processa a cobrança no pagamentos-service (:3002) e, por fim, grava o pedido e publica o evento 'PedidoCriado'.

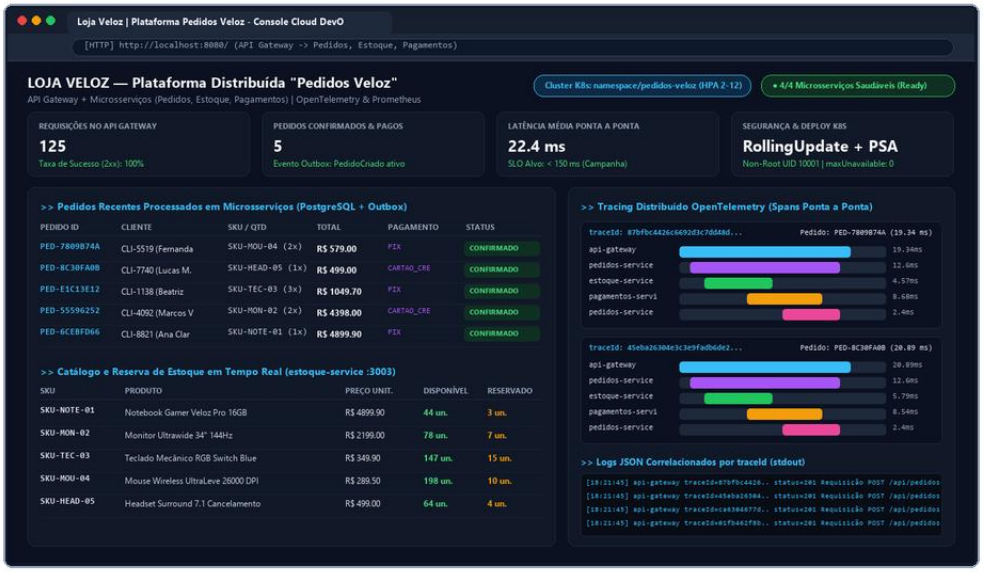


Figura 3 — Console Operacional da Loja Veloz no API Gateway (http://localhost:8080) exibindo pedidos pagos, baixa de estoque e traces.

4. Kubernetes em Produção Mínima (Manifests, Probes e Segurança)

Na pasta k8s/, organizei os manifestos numerados de 00 a 06 para refletir a ordem de aplicação no cluster de produção. Abaixo estão as principais configurações implementadas para garantir estabilidade e segurança:

- **Namespace com Pod Security Admission (00-namespace.yaml):** O namespace 'pedidos-veloz' aplica o rótulo nativo pod-security.kubernetes.io/enforce: restricted, impedindo que qualquer container suba como root ou peça privilégios de administrador no nó.
- **Desacoplamento com ConfigMap e Secret (01 e 02):** As URLs internas do cluster (ex: http://estoque-service.pedidos-veloz.svc.cluster.local:3003) ficam no ConfigMap, enquanto a senha do PostgreSQL e a chave da API de pagamentos ficam isoladas no objeto Secret.
- **Deployments com Probes e SecurityContext (03-deployments.yaml):** Todos os Deployments definem readinessProbe (GET /ready) e livenessProbe (GET /health), limites de CPU/Memória, além de securityContext com runAsNonRoot: true, runAsUser: 10001, readOnlyRootFilesystem: true e drop: ['ALL'].
- **Services, Ingress e NetworkPolicy Zero-Trust (04 e 06):** Os serviços internos usam tipo ClusterIP (inacessíveis de fora do cluster) e apenas o api-gateway é exposto via Ingress. Além disso, a NetworkPolicy garante que somente o pedidos-service e o gateway possam abrir conexão com estoque e pagamentos.

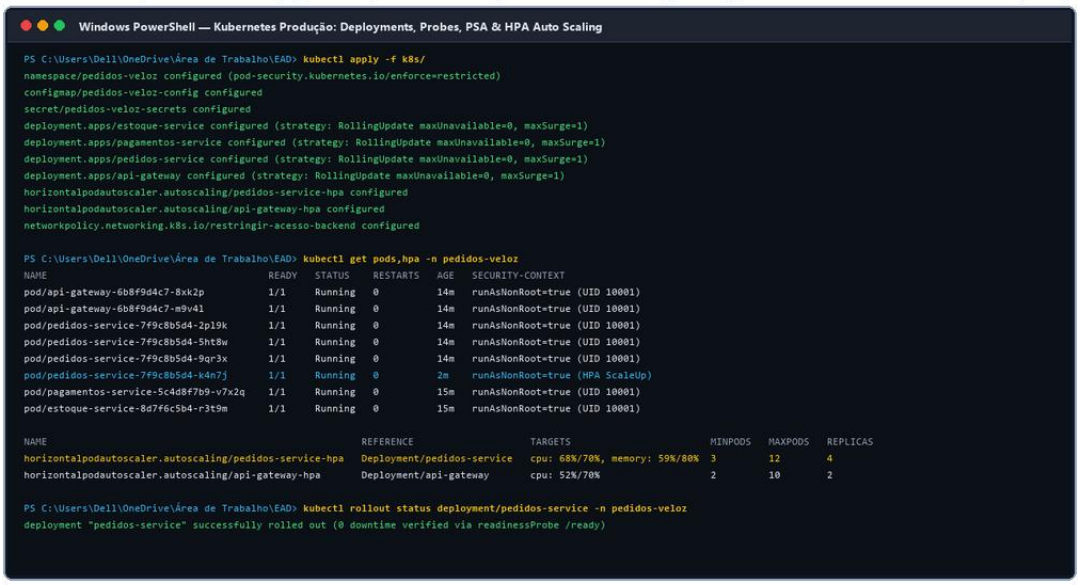


Figura 4 — Aplicação dos manifestos Kubernetes, verificação dos Pods rodando como não-root (UID 10001), HPA ativo e Rollout sem downtime.

Por que as Probes evitam indisponibilidade durante o Deploy?
Antes desta modernização, a Loja Veloz sofria com erros 502 durante atualizações porque o balanceador mandava clientes para containers que ainda estavam carregando. Com a readinessProbe apontando para /ready e a estratégia RollingUpdate (maxUnavailable: 0, maxSurge: 1), o Kubernetes só envia tráfego para o Pod novo quando ele responde 200 OK.

5. Pipeline de CI/CD Automatizado no GitHub Actions

Para automatizar a entrega do código até a produção, criei o pipeline .github/workflows/ci-cd.yaml no GitHub Actions. Ele foi estruturado em quatro estágios sequenciais (jobs dependentes via 'needs'), funcionando como portões de qualidade (quality gates) que travam o deploy caso qualquer verificação falhe:

- **Estágio 1 — Lint, K8s Schema & Terraform Validate:** Verifica a sintaxe dos 4 microsserviços (npm run lint), valida se todos os manifestos da pasta k8s/ estão em conformidade com a API oficial do Kubernetes usando Kubeconform e roda 'terraform fmt -check' e 'terraform validate' na infraestrutura.
- **Estágio 2 — Testes Automatizados (Unitários + E2E):** Executa toda a bateria de testes de reserva de estoque, aprovação/recusa de pagamento antifraude, criação de pedidos e propagação do traceId pelo API Gateway.
- **Estágio 3 — Build Multi-Stage, Scan Trivy & Publicação no Registry:** Usa matriz paralela para construir as imagens dos 4 microsserviços, aplica duas tags (:1.2.0 e :sha-commit), executa o scanner Aqua Trivy (configurado com exit-code: 1 para bloquear vulnerabilidades CRITICAL ou HIGH) e faz push para o GitHub Container Registry (ghcr.io) autenticando via secrets.GITHUB_TOKEN.
- **Estágio 4 — Deploy Contínuo (CD) no Kubernetes:** Disparado apenas na branch main após aprovação dos estágios anteriores. Autentica no cluster usando o segredo secrets.KUBE_CONFIG, aplica os manifestos atualizados, troca a tag da imagem e monitora o 'kubectl rollout status' até concluir.

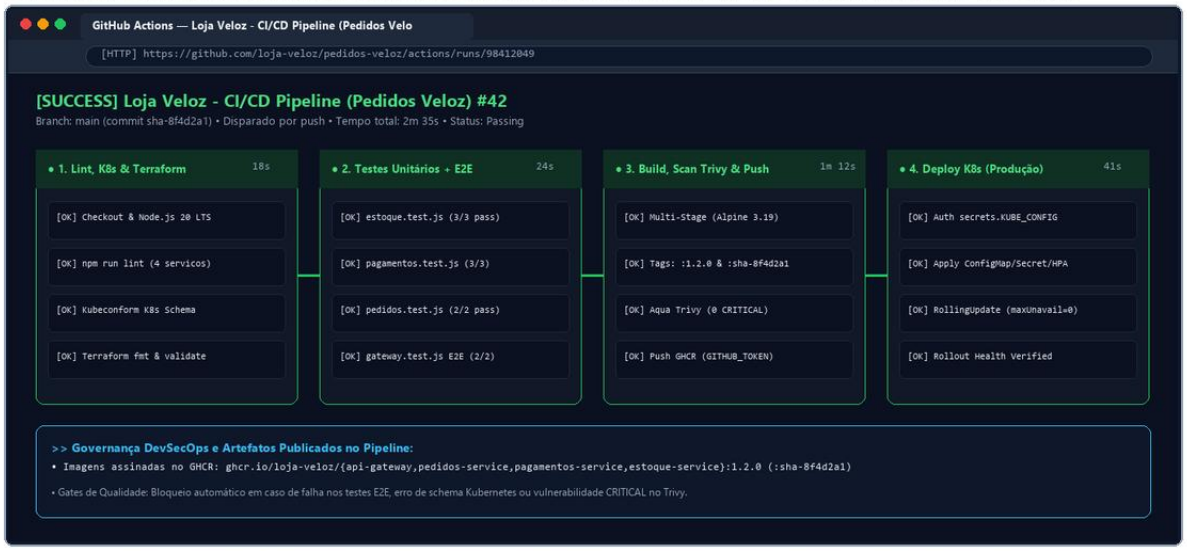


Figura 5 — Execução do pipeline CI/CD no GitHub Actions com os 4 jobs aprovados, scan Trivy sem CVEs críticos e deploy concluído.

Segredo Utilizado no Pipeline	Onde é Configurado	Finalidade Prática na Automação
secrets.GITHUB_TOKEN	GitHub Actions Token Nativo	Autenticar no GitHub Container Registry (ghcr.io) e publicar as imagens Docker versionadas.
secrets.KUBE_CONFIG	GitHub Environment Secrets	Conectar de forma criptografada ao cluster Kubernetes de produção para executar o rollout.
pedidos-veloz-secrets	Kubernetes Secret / Vault	Injetar POSTGRES_PASSWORD e PAYMENT_PROVIDER_API_KEY dentro dos Pods em execução.

6. Observabilidade na Prática: Métricas, Logs e Tracing Distribuído

Na pasta observability/, configurei o Prometheus (prometheus.yml) para coletar a rota /metrics dos 4 microsserviços a cada 15 segundos e o OpenTelemetry Collector (otel-collector-config.yaml) para receber os traces via protocolo OTLP e enviá-los ao Jaeger. Todos os microsserviços propagam os cabeçalhos HTTP 'x-trace-id' e 'traceparent' (padrão W3C). Dessa forma, quando um pedido é criado, conseguimos inspecionar no Jaeger a cascata completa de spans e cruzar o mesmo traceId nos logs JSON.

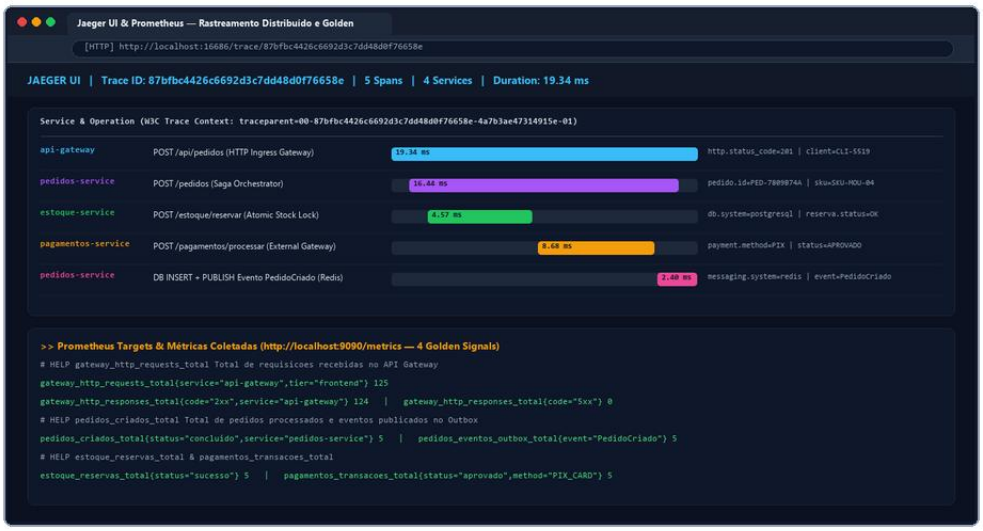


Figura 6 — Visualização do Trace Distribuído no Jaeger UI (5 spans atravessando 4 serviços em 19.34 ms) e métricas no Prometheus.

7. Estratégia de Deploy, Escalabilidade (HPA) e IaC com Terraform

Para fechar a solução fim a fim, definimos como a plataforma se comporta durante o pico da campanha promocional e como toda a infraestrutura de nuvem é provisionada:

- **Deploy RollingUpdate + Evolução para Canary:** Nos Deployments (03-deployments.yaml), configuramos RollingUpdate com maxUnavailable: 0 e maxSurge: 1 para garantir zero-downtime imediato no MVP. Para atualizações futuras de alto risco na regra de cobrança do pagamentos-service, a arquitetura já suporta deploy Canary via Ingress/Service Mesh direcionando 10% do tráfego inicialmente.
- **Escalabilidade Horizontal com HPA v2 (05-hpa.yaml):** O pedidos-service escala automaticamente de 3 até 12 réplicas (e o api-gateway de 2 até 10 réplicas) quando o uso médio de CPU atinge 70% ou memória atinge 80%. Configuramos uma janela de estabilização (stabilizationWindowSeconds: 300) no scaleDown para evitar que os Pods sejam desligados cedo demais entre ondas de acesso da promoção.
- **Infraestrutura como Código com Terraform (pasta terraform/):** Criamos os arquivos providers.tf, variables.tf, main.tf e outputs.tf que provisionam de forma reproduzível a VPC isolada (10.20.0.0/16) com sub-redes privadas em múltiplas zonas de disponibilidade, o cluster Kubernetes gerenciado (Amazon EKS 1.30) e a instância do banco de dados Amazon RDS PostgreSQL 16 Multi-AZ com armazenamento criptografado.

Conclusão dos Resultados Alcançados para a Loja Veloz

Com a implementação entregue neste repositório, a Loja Veloz passa a contar com ambiente local de 1 comando no Docker Compose, imagens Alpine seguras rodando sem root, pipeline CI/CD com testes e scan Trivy, deploy sem quedas no Kubernetes com auto-scaling (HPA) e rastreabilidade total via OpenTelemetry, Jaeger e Prometheus.